



Shri Vile Parle Kelavani Mandal's
INSTITUTE OF TECHNOLOGY
DHULE (M.S.)
DEPARTMENT OF COMPUTER ENGINEERING

Subject : Machine Learning Lab	Remark
Name: _____ Roll No. : _____	
Class: T. Y. Comp. Engg. Batch : T Division: _____	Signature
Expt. No. : 02 Performance Date : / /	
Title : Write a program to Perform Data Preprocessing through various feature-engineering algorithms by python.	

Rubrics :	Date of Submission:		
	Marks Split up to	Maximum Marks	Marks Obtained
	Performance/conduction		
	Report Writing		
	Attendance		
	Viva/Oral		
	Total Marks		
	Aim :	The main aim of this lab is to demonstrate the data preprocessing techniques used in machine learning using Python The secondary objectives are: 1. to explain how a deal with missing data, 2. how to normalize data 3. how to split data into train and test dataset 4. how to deal with categorical data. 5. Rescaling of data	
Software Required :	Jupyter Notebook , Python 3.6, Or Google Colab		
Dataset :	Data.csv		

Theory :

Data Preprocessing : Concepts

Data is truly considered a resource in today's world. As per the World Economic Forum, by 2025 we will be generating about 463 exabytes of data globally per day! But is all this data fit enough to be used by machine learning algorithms? How do we decide that? In this article we will explore the topic of data preprocessing — transforming the data such that it becomes machine-readable...

What is Data Preprocessing?

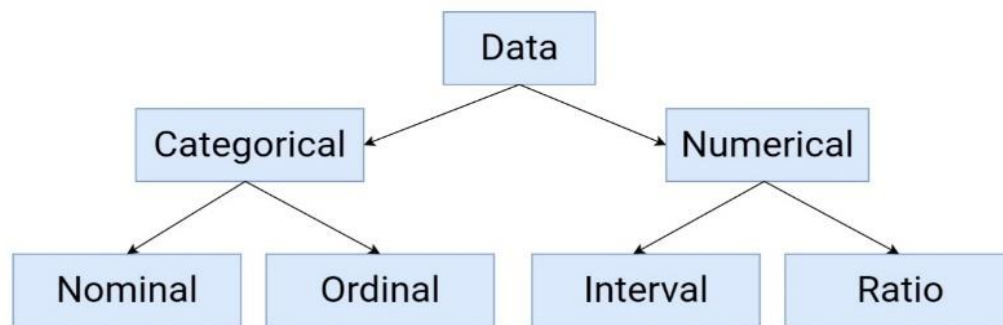
In any Machine Learning process, Data Preprocessing is that step in which the data gets transformed, or Encoded, to bring it to such a state that now the machine can easily parse it. In other words, the features of the data can now be easily interpreted by the algorithm.

Features in Machine Learning

A dataset can be viewed as a collection of data objects, which are often also called as a records, points, vectors, patterns, events, cases, samples, observations, or entities. Data objects are described by a number of features, that capture the basic characteristics of an object, such as the mass of a physical object or the time at which an event occurred, etc.. Features are often called as variables, characteristics, fields, attributes, or dimensions.

“A feature is an individual measurable property or characteristic of a phenomenon being observed”

For instance, color, mileage and power can be considered as features of a car. There are different types of features that we can come across when we deal with data.



Nominal	Ordinal	Interval	Ratio
Categorical variables without any implied order	Categorical variables with a natural implied order but the scale of difference is not defined	Numeric variables with a defined unit of measurement, so the differences between values are meaningful	Numeric variables with a defined unit of measurement but both differences and ratios are meaningful
Example : A new car model comes in these colors : Black, Blue, White, Silver	Example : Sizes of clothes has a natural order : Extra Small < Small < Medium < Large < Extra Large - But this does not mean Large - Medium = Medium - Small	Examples : Calender Dates, Temperature in Celsius or Fahrenheit	Examples : Temperature in Kelvin, Monetary quantities, Counts, Age, Mass, Length, Electrical Current

Generally they are :

- Data Quality Assessment
- Feature Aggregation
- Feature Sampling
- Dimensionality Reduction
- Feature Encoding

Data Quality Assessment

Because data is often taken from multiple sources which are normally not too reliable and that too in different formats, more than half our time is consumed in dealing with data quality issues when working on a machine learning problem. It is simply unrealistic to expect that the data will be perfect. There may be problems due to human error, limitations of measuring devices, or flaws in the data collection process. Let's go over a few of them and methods to deal with them :

1.Missing values :

It is very much usual to have missing values in your dataset. It may have happened during data collection, or maybe due to some data validation rule, but regardless missing values must be taken into consideration.

a. Eliminate rows with missing data :

Simple and sometimes effective strategy. Fails if many objects have missing values. If a feature has mostly missing values, then that feature itself can also be eliminated.

b. Estimate missing values :

If only a reasonable percentage of values are missing, then we can also run simple interpolation methods to fill in those values. However, most common method of dealing with missing values is by filling them in with the mean, median or mode value of the respective feature.

2. Inconsistent values :

We know that data can contain inconsistent values. Most probably we have already faced this issue at some point. For instance, the 'Address' field contains the 'Phone number'. It may be due to human error or maybe the information was misread while being scanned from a handwritten form.

It is therefore always advised to perform data assessment like knowing what the data type of the features should be and whether it is the same for all the data objects.

3. Duplicate values :

A dataset may include data objects which are duplicates of one another. It may happen when say the same person submits a form more than once. The term deduplication is often used to refer to the process of dealing with duplicates.

In most cases, the duplicates are removed so as to not give that particular data object an advantage or bias, when running machine learning algorithms.

Feature Aggregation

Feature Aggregations are performed so as to take the aggregated values in order to put the data in a better perspective. Think of transactional data, suppose we have day-to-day transactions of a product from recording the daily sales of that product in various store locations over the year. Aggregating the transactions to single store-wide monthly or yearly transactions will help us reducing hundreds or potentially thousands of transactions that occur daily at a specific store, thereby reducing the number of data objects.

This results in reduction of memory consumption and processing time

Aggregations provide us with a high-level view of the data as the behavior of groups or aggregates is more stable than individual data objects

Feature Sampling

Sampling is a very common method for selecting a subset of the dataset that we are analyzing. In most cases, working with the complete dataset can turn out to be too expensive considering the memory and time constraints. Using a sampling algorithm can help us reduce the size of the dataset to a point where we can use a better, but more expensive, machine learning algorithm.

The key principle here is that the sampling should be done in such a manner that the sample generated should have approximately the same properties as the original dataset, meaning that the sample is representative. This involves choosing the correct sample size and sampling strategy.

Simple Random Sampling dictates that there is an equal probability of selecting any particular entity. It has two main variations as well :

- **Sampling without Replacement** : As each item is selected, it is removed from the set of all the objects that form the total dataset.

- **Sampling with Replacement** : Items are not removed from the total dataset after getting selected. This means they can get selected more than once.

Although Simple Random Sampling provides two great sampling techniques, it can fail to output a representative sample when the dataset includes object types which vary drastically in ratio. This can cause problems when the sample needs to have a proper representation of all object types, for example, when we have an imbalanced dataset.

Feature Encoding

As mentioned before, the whole purpose of data preprocessing is to encode the data in order to bring it to such a state that the machine now understands it.

Feature encoding is basically performing transformations on the data such that it can be easily accepted as input for machine learning algorithms while still retaining its original meaning.

There are some general norms or rules which are followed when performing feature encoding. For Continuous variables :

- **Nominal** : Any one-to-one mapping can be done which retains the meaning. For instance, a permutation of values like in One-Hot Encoding.

	Name	Generation	Gen 1	Gen 2	Gen 3	Gen 4	Gen 5
4	Octillery	Gen 2	0	1	0	0	0
5	Helioptile	Gen 6	0	0	0	0	0
6	Dialga	Gen 4	0	0	0	1	0
7	DeoxysDefense Forme	Gen 3	0	0	1	0	0
8	Rapidash	Gen 1	1	0	0	0	0
9	Swanna	Gen 5	0	0	0	0	1

- **Ordinal** : An order-preserving change of values. The notion of small, medium and large can be represented equally well with the help of a new function, that is, $\langle \text{new_value} = f(\text{old_value}) \rangle$ - For example, $\{0, 1, 2\}$ or maybe $\{1, 2, 3\}$.

Numeric variables:

- **Interval** : Simple mathematical transformation like using the equation $\langle \text{new_value} = a * \text{old_value} + b \rangle$, a and b being constants. For example, Fahrenheit and Celsius scales, which differ in their Zero values size of a unit, can be encoded in this manner.
- **Ratio** : These variables can be scaled to any particular measures, of course while still maintaining the meaning and ratio of their values. Simple mathematical

transformations work in this case as well, like the transformation $\text{new_value} = a * \text{old_value}$. For, length can be measured in meters or feet, money can be taken in different currencies.

Train / Validation / Test Split

After feature encoding is done, our dataset is ready for the exciting machine learning algorithms!

But before we start deciding the algorithm which should be used, it is always advised to split the dataset into 2 or sometimes 3 parts. Machine Learning algorithms, or any algorithm for that matter, has to be first trained on the data distribution available and then validated and tested, before it can be deployed to deal with real-world data.

Training data : This is the part on which your machine learning algorithms are actually trained to build a model. The model tries to *learn* the dataset and its various characteristics and intricacies, which also raises the issue of Overfitting v/s Underfitting.

Validation data : This is the part of the dataset which is used to validate our various model fits. In simpler words, we use validation data to choose and improve our model hyperparameters. The model does not *learn* the validation set but uses it to get to a better state of hyperparameters.

Test data : This part of the dataset is used to test our model hypothesis. It is left untouched and unseen until the model and hyperparameters are decided, and only after that the model is applied on the test data to get an accurate measure of how it would

Procedure:



perform when deployed on real-world data.

Split Ratio : Data is split as per a split ratio which is highly dependent on the type of model we are building and the dataset itself. If our dataset and model are such that a lot of training is required, then we use a larger chunk of the data just for training purposes (usually the case) — For instance, training on textual data, image data, or video data usually involves thousands of features!

If the model has a lot of hyperparameters that can be tuned, then keeping a higher percentage of data for the validation set is advisable. Models with less number of hyperparameters are easy to tune and update, and so we can keep a smaller validation set. Like many other things in Machine Learning, the split ratio is highly dependent on the problem we are trying to solve and must be decided after taking into account all the various details about the model and the dataset in hand.

Steps in Data Preprocessing in Machine Learning

There are seven significant steps in data preprocessing in Machine Learning:

1. Acquire the dataset

To build and develop Machine Learning models, you must first acquire the relevant dataset. This dataset will be comprised of data gathered from multiple and disparate sources which are then combined in a proper format to form a dataset. Dataset formats differ according to use cases. For instance, a business dataset will be entirely different from a medical dataset. While a business dataset will contain relevant industry and business data, a medical dataset will include healthcare-related data.

There are several online sources from where you can download datasets like <https://www.kaggle.com/uciml/datasets> and <https://archive.ics.uci.edu/ml/index.php>. You can also create a dataset by collecting data via different Python APIs. Once the dataset is ready, you must put it in a CSV, or HTML, or XLSX file formats.

2. Import all the crucial libraries

Since Python is the most extensively used and also the most preferred library by Data Scientists around the world, we'll show you how to import Python libraries for data preprocessing in Machine Learning. Read more about [Python libraries for Data Science here](#). The predefined Python libraries can perform specific data preprocessing jobs. The three core Python libraries used for this data preprocessing in Machine Learning are:

- **NumPy** – NumPy is the fundamental package for scientific calculation in Python. Hence, it is used for inserting any type of mathematical operation in the code. Using NumPy, you can also add large multidimensional arrays and matrices in your code.
-
- **Pandas** – Pandas is an excellent open-source Python library for data manipulation and analysis. It is extensively used for importing and managing the datasets. It packs in high-performance, easy-to-use data structures and data analysis tools for Python.
- **Matplotlib** – Matplotlib is a Python 2D plotting library that is used to plot any type of charts in Python. It can deliver publication-quality figures in numerous hard copy formats and interactive environments across platforms (IPython shells, Jupyter notebook, web application servers, etc.).

3. Import the dataset

In this step, you need to import the dataset/s that you have gathered for the ML project at hand. However, before you can import the dataset/s, you must set the current directory as the working directory. You can set the working directory in Spyder IDE in three simple steps:

- I. Save your Python file in the directory containing the dataset.
- II. Go to File Explorer option in Spyder IDE and choose the required directory.
- III. Now, click on the F5 button or Run option to execute the file.

Once you've set the working directory containing the relevant dataset, you can import the dataset using the "read_csv()" function of the Pandas library. This function can read a CSV file (either locally or through a URL) and also perform various operations on it. The read_csv() is written as:

```
data_set= pd.read_csv('Dataset_name.csv/path/url')
```

In this line of code, “data_set” denotes the name of the variable wherein you stored the dataset. The function contains the name of the dataset as well. Once you execute this code, the dataset will be successfully imported.

During the dataset importing process, there’s another essential thing you must do – extracting dependent and independent variables. For every Machine Learning model, it is necessary to separate the independent variables (matrix of features) and dependent variables in a dataset.

How to extract the independent variables?

To extract the independent variables, you can use “iloc[]” function of the Pandas library. This function can extract selected rows and columns from the dataset.

```
x= data_set.iloc[:, :-1].values
```

In the line of code above, the first colon(:) considers all the rows and the second colon(:) considers all the columns. The code contains “:-1” since you have to leave out the last column containing the dependent variable.

How to extract the dependent variable?

You can use the “iloc[]” function to extract the dependent variable as well. Here’s how you write it:

```
y= data_set.iloc[:, -1].values
```

This line of code considers all the rows with the last column only.

4. Identifying and handling the missing values

In data preprocessing, it is pivotal to identify and correctly handle the missing values, failing to do this, you might draw inaccurate and faulty conclusions and inferences from the data. Needless to say, this will hamper your ML project.

Basically, there are two ways to handle missing data:

Deleting a particular row – In this method, you remove a specific row that has a null value for a feature or a particular column where **more than 75%** of the values are missing. However, this **method is not 100% efficient**, and it is recommended that you use it only when the dataset has adequate samples. You must ensure that after deleting the data, there remains no addition of bias.

Calculating the mean – This method is useful for features having numeric data like age, salary, year, etc. Here, you can calculate the mean, median, or mode of a particular feature or column or row that contains a missing value and replace the result for the missing value. This method can add variance to the dataset, and any loss of data can be efficiently negated. Hence, it yields better results compared to the first method (omission)

of rows/columns). Another way of approximation is through the deviation of neighbouring values. However, this works best for linear data.

5. Encoding the categorical data

Categorical data refers to the information that has specific categories within the dataset.

The labelEncoder and OneHotEncoder only works on categorical features.

Label Encoding refers to converting the labels into numeric form so as to convert it into the machine-readable form. Machine learning algorithms can then decide in a better way on how those labels must be operated. It is an important pre-processing step for the structured dataset in supervised learning.

We need first to extract the categorical features using boolean mask.

```
# Categorical boolean mask
categorical_feature_mask = X.dtypes==object# filter categorical columns
using mask and turn it into a list
categorical_cols = X.columns[categorical_feature_mask].tolist()
```

LabelEncoder converts each class under specified feature to a numerical value. Let's go through the steps to see how to do it.

Instantiate a LabelEncoder object:

```
# import labelencoder
from sklearn.preprocessing import LabelEncoder# instantiate
labelencoder object
le = LabelEncoder()
```

Apply LabelEncoder on each of the categorical columns:

```
# apply le on categorical feature columns
X[categorical_cols] = X[categorical_cols].apply(lambda col:
le.fit_transform(col))X[categorical_cols].head(10)
```

Limitation of label Encoding

Label encoding convert the data in machine readable form, but it assigns a unique number(starting from 0) to each class of data. This may lead to the generation of priority issue in training of data sets. A label with high value may be considered to have high priority than a label having lower value.

Thus allowing model learning this result will lead to poor performance. Therefore, for dataframe containing multi class features, a further step of OneHotEncoder is needed. Let's see the steps to do it.

Instantiate OneHotEncoder object:

```
# import OneHotEncoder
from sklearn.preprocessing import OneHotEncoder# instantiate
OneHotEncoder
```

```
ohe = OneHotEncoder(categorical_features = categorical_feature_mask,
sparse=False )
# categorical_features = boolean mask for categorical columns
# sparse = False output an array not sparse matrix
```

We need to specify categorical feature using its mask inside OneHotEncoder. The `sparse=False` argument outputs a non-sparse matrix.

Apply OneHotEncoder on DataFrame:

```
# apply OneHotEncoder on categorical feature columns
X_ohe = ohe.fit_transform(X) # It returns an numpy array
```

Note that the output is a numpy array, not a dataframe. For each class under a categorical feature, a new column is created for it.

6. Splitting the dataset

Every dataset for Machine Learning model must be split into two separate sets – training set and test set.

Training set denotes the subset of a dataset that is used for training the machine learning model. Here, you are already aware of the output. A test set, on the other hand, is the subset of the dataset that is used for testing the machine learning model. The ML model uses the test set to predict outcomes.

Usually, the dataset is split into 70:30 ratio or 80:20 ratio. This means that you either take 70% or 80% of the data for training the model while leaving out the rest 30% or 20%. The splitting process varies according to the shape and size of the dataset in question.

To split the dataset, you have to write the following line of code –

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test= train_test_split(x, y, test_size=
0.2, random_state=0)
```

Here, the first line splits the arrays of the dataset into random train and test subsets. The second line of code includes four variables:

- `x_train` – features for the training data
- `x_test` – features for the test data
- `y_train` – dependent variables for training data
- `y_test` – independent variable for testing data

Thus, the `train_test_split()` function includes four parameters, the first two of which are for arrays of data. The `test_size` function specifies the size of the test set. The `test_size` maybe .5, .3, or .2 – this specifies the dividing ratio between the training and test sets.

The last parameter, “random_state” sets seed for a random generator so that the output is always the same.

7. Feature scaling

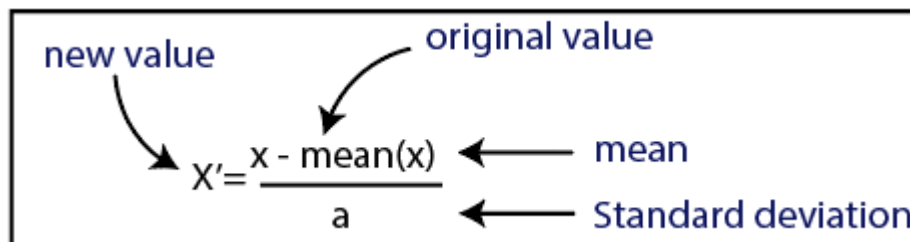
Feature scaling marks the end of the **data preprocessing in Machine Learning**. It is a method to standardize the independent variables of a dataset within a specific range. In other words, feature scaling limits the range of variables so that you can compare them on common grounds. Thus, you must remove this issue by performing feature scaling for Machine Learning.

Most ML models are based on Euclidean Distance, which is represented as:

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

You can perform feature scaling in Machine Learning in two ways:

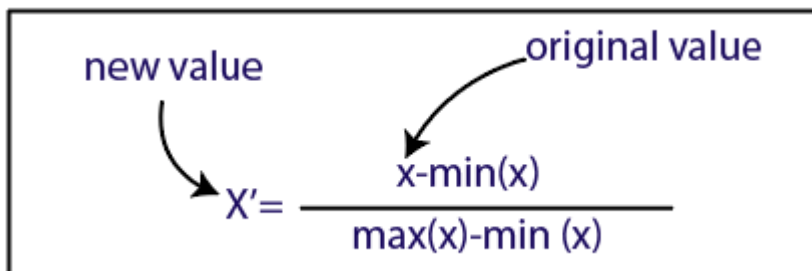
1. **Standardize Data:** Standardization is a useful technique to transform attributes with a Gaussian distribution and differing means and standard deviations to a standard Gaussian distribution with a mean of 0 and a standard deviation of 1. We can standardize data using scikit-learn with the StandardScaler class



A diagram showing the formula for standardization: $X' = \frac{x - \text{mean}(x)}{a}$. Arrows point from labels to parts of the formula: 'new value' points to X' , 'original value' points to x , 'mean' points to $\text{mean}(x)$, and 'Standard deviation' points to a .

2. Rescale Data

- When our data is comprised of attributes with varying scales, many machine learning algorithms can benefit from rescaling the attributes to all have the same scale.
- This is useful for optimization algorithms in used in the core of machine learning algorithms like gradient descent.
- It is also useful for algorithms that weight inputs like regression and neural networks and algorithms that use distance measures like K-Nearest Neighbors.
- We can rescale your data using scikit-learn using the **MinMaxScaler** class.



A diagram showing the formula for MinMax scaling: $X' = \frac{x - \min(x)}{\max(x) - \min(x)}$. Arrows point from labels to parts of the formula: 'new value' points to X' , 'original value' points to x , and the denominator $\max(x) - \min(x)$ is shown below the fraction line.

Practice Problems: Conclusion: Test Your Knowledge	Download the dataset and perform above 7 steps for data preprocessing on it In this way successfully implemented data preprocessing technique on our dataset 1. When to use the Label Encoder and When to use OneHotEncoder 2. Write a code to Feature Scaling our dataset numerical variable using MinMaxScaler 3. For the below student dataset remove missing values from column 'gender' and 'marks' <pre> import pandas as pd import numpy as np students = [[85, 'M', 'verygood'], [95, 'F', 'excellent'], [75, None, 'good'], [np.NaN, 'M', 'average'], [70, 'M', 'good'], [np.NaN, None, 'verygood'], [92, 'F', 'verygood'], [98, 'M', 'excellent']] dfstd = pd.DataFrame(students) dfstd.columns = ['marks', 'gender', 'result'] (Attach separate sheet for answer) </pre> 4. For dataset given in above Apply the proper Categorical Encoder to encode column 'gender' and 'result' (Attach separate sheet for answer)
--	--

